```
library(arrow)
library(data.table)
library(ggplot2)
library(foreach)
library(doParallel)
library(purrr)
library(lubridate)
library(bench)
library(scales)

set.seed(47)

options(scipen = 999)

setwd(dirname(rstudioapi::getActiveDocumentContext()$path))

data_dir = "data"

output_dir = "output"

# task 1

trip_files <- sort(list.files(data_dir, pattern = "^yellow_tripdata_.*\\.parquet$",
                              full.names = TRUE))
zone_file  <- file.path(data_dir, "taxi_zone_lookup.csv")

stopifnot(
  "No yellow_tripdata_*.parquet files found in data/" = length(trip_files) > 0,
  "taxi_zone_lookup.csv not found in data/"            = file.exists(zone_file)
)

read_one_month <- function(path) {
  dt <- as.data.table(arrow::read_parquet(path))
  dt[, src_file := basename(path)]
  dt
}

trip_list <- lapply(trip_files, read_one_month)
names(trip_list) <- sub("\\.parquet$", "", basename(trip_files))

trips <- rbindlist(trip_list, use.names = TRUE, fill = TRUE)
cat("Combined rows:", format(nrow(trips), big.mark = ","), "\n")
cat("Combined cols:", ncol(trips), "\n")

# 1.2 Inspect

str(trips)
dim(trips)
summary(trips[, .(trip_distance, fare_amount, tip_amount, total_amount, passenger_count)])

# 1.3 Handle missing/duplicate/invalid records

n0 <- nrow(trips)

trips[, `:=`(
  tpep_pickup_datetime  = as.POSIXct(tpep_pickup_datetime,  tz = "UTC"),
  tpep_dropoff_datetime = as.POSIXct(tpep_dropoff_datetime, tz = "UTC")
)]

trips <- unique(trips)
n_dup <- n0 - nrow(trips)

trips <- trips[
  !is.na(passenger_count) &
    !is.na(tpep_pickup_datetime) & !is.na(tpep_dropoff_datetime) &
```

```
    !is.na(tpep_pickup_datetime) & !is.na(tpep_dropoff_datetime) &
    fare_amount > 0 & total_amount > 0 &
    trip_distance > 0 &
    tpep_dropoff_datetime > tpep_pickup_datetime &
    as.numeric(tpep_dropoff_datetime - tpep_pickup_datetime, units = "hours") <= 24 &
    PULocationID %between% c(1, 263) & DOLocationID %between% c(1, 263)
]

n_invalid <- n0 - n_dup - nrow(trips)

cat(sprintf("Start rows:           %s\n", format(n0, big.mark = ",")))
cat(sprintf("Duplicates removed:   %s\n", format(n_dup, big.mark = ",")))
cat(sprintf("Invalid rows removed: %s\n", format(n_invalid, big.mark = ",")))
cat(sprintf("Final rows:           %s (%.1f%% retained)\n",
            format(nrow(trips), big.mark = ","), 100 * nrow(trips) / n0))

# 1.4 Data type conversion

trips[, `:=`(
  VendorID     = factor(VendorID),
  RatecodeID   = factor(RatecodeID),
  payment_type = factor(payment_type,
                        levels = 1:6,
                        labels = c("Credit card", "Cash", "No charge",
                                   "Dispute", "Unknown", "Voided trip")),
  passenger_count = as.integer(passenger_count)
)]

# 1.5 Extract temporal attributes

trips[, `:=`(
  pickup_hour  = hour(tpep_pickup_datetime),
  pickup_day   = mday(tpep_pickup_datetime),
  pickup_dow   = wday(tpep_pickup_datetime, label = TRUE, abbr = TRUE, week_start = 1),
  pickup_month = month(tpep_pickup_datetime, label = TRUE, abbr = TRUE),
  trip_minutes = as.numeric(tpep_dropoff_datetime - tpep_pickup_datetime, units = "mins")
)]

# 1.6 Drop unneeded columns

drop_cols <- intersect(c("store_and_fwd_flag", "src_file"), names(trips))
trips[, (drop_cols) := NULL]
cat("Memory after cleanup/drop:", format(object.size(trips), units = "MB"), "\n")

# 1.7 Import taxi zone lookup table

zones <- fread(zone_file, showProgress = FALSE)
setnames(zones, old = names(zones), new = make.unique(names(zones)))
print(head(zones, 5))

# 1.8 Join trips with zone info

setkey(zones, LocationID)

trips <- zones[, .(LocationID, PUBorough = Borough, PUZone = Zone)][
  trips, on = c("LocationID" = "PULocationID")]
setnames(trips, "LocationID", "PULocationID")

trips <- zones[, .(LocationID, DOBorough = Borough, DOZone = Zone)][
  trips, on = c("LocationID" = "DOLocationID")]
setnames(trips, "LocationID", "DOLocationID")

setcolorder(trips, c("tpep_pickup_datetime", "tpep_dropoff_datetime",
                     "PULocationID", "PUBorough", "PUZone",
                     "DOLocationID", "DOBorough", "DOZone"))

print(head(trips[, .(tpep_pickup_datetime, PUBorough, PUZone, DOBorough, DOZone,
```

```
                      fare_amount, total_amount)], 5))

fwrite(trips, file.path(output_dir, "trips_clean.csv"))
cat("Clean, joined dataset:", format(nrow(trips), big.mark = ","), "rows x",
    ncol(trips), "cols\n")

# task 2

# 2.1 Trips by hour of day
by_hour <- trips[, .(trips = .N), by = pickup_hour][order(pickup_hour)]
print(by_hour)

# 2.2 Demand across days of week
by_dow <- trips[, .(trips = .N), by = pickup_dow][order(pickup_dow)]
print(by_dow)

# 2.3 Monthly variation
by_month <- trips[, .(trips = .N), by = pickup_month][order(pickup_month)]
print(by_month)

# 2.4 Average fare & total revenue by time period (hour)
fare_by_hour <- trips[, .(avg_fare = mean(fare_amount),
                          total_revenue = sum(total_amount),
                          trips = .N), by = pickup_hour][order(pickup_hour)]
print(fare_by_hour)

# 2.5 Most frequently travelled routes
top_routes <- trips[, .(trips = .N), by = .(PUZone, DOZone)][order(-trips)][1:10]
print(top_routes)

# 2.6 Highest-revenue routes
top_rev_routes <- trips[, .(total_revenue = sum(total_amount), trips = .N),
                        by = .(PUZone, DOZone)][order(-total_revenue)][1:10]
print(top_rev_routes)

# 2.7 Trip distance vs. fare amount
dist_fare_corr <- cor(trips$trip_distance, trips$fare_amount)
cat(sprintf("Correlation(trip_distance, fare_amount) = %.3f\n", dist_fare_corr))

dist_fare_summary <- trips[, .(avg_fare = mean(fare_amount),
                               avg_distance = mean(trip_distance)),
                           by = .(distance_bin = cut(trip_distance,
                                                     breaks = c(0,1,2,5,10,20,Inf)))]
print(dist_fare_summary[order(distance_bin)])

# 2.8 Payment methods by borough
payment_borough <- trips[, .(trips = .N), by = .(PUBorough, payment_type)]
payment_borough[, pct := round(100 * trips / sum(trips), 1), by = PUBorough]
print(payment_borough[order(PUBorough, -trips)])

# 2.9 Additional pattern: tip percentage by hour
trips[, tip_pct := fifelse(fare_amount > 0, 100 * tip_amount / fare_amount, NA_real_)]
tip_by_hour <- trips[, .(avg_tip_pct = mean(tip_pct, na.rm = TRUE)), by = pickup_hour][order(pick
print(tip_by_hour)

# task 3

hours_vec <- trips$pickup_hour
fare_vec  <- trips$fare_amount
hour_levels <- sort(unique(hours_vec))

# 3.1 Functional (lapply / purrr::map)
avg_fare_lapply <- function(hours, fares, levels) {
  split_fares <- split(fares, hours)
  unlist(lapply(split_fares, mean))[as.character(levels)]
}
avg_fare_purrr <- function(hours, fares, levels) {
```

```
  split_fares <- split(fares, hours)
  purrr::map_dbl(split_fares, mean)[as.character(levels)]
}
res_lapply <- avg_fare_lapply(hours_vec, fare_vec, hour_levels)
res_purrr  <- avg_fare_purrr(hours_vec, fare_vec, hour_levels)

# 3.2 Vectorized base R (tapply)
avg_fare_vectorized <- function(hours, fares) tapply(fares, hours, mean)
res_vectorized <- avg_fare_vectorized(hours_vec, fare_vec)

# 3.3 data.table
avg_fare_dt <- function(dt) dt[, .(avg_fare = mean(fare_amount)), by = pickup_hour][order(pickup_
res_dt <- avg_fare_dt(trips)

# 3.4 Correctness check
cat("All three agree:",
    isTRUE(all.equal(as.numeric(res_lapply), res_dt$avg_fare, check.attributes = FALSE)), "\n")

# task 4

# 4.1 Month-wise partitions
month_splits <- split(trips, trips$pickup_month)
month_splits <- month_splits[sapply(month_splits, nrow) > 0]
cat("Partitions:", paste(names(month_splits), collapse = ", "), "\n")
print(sapply(month_splits, nrow))

# 4.2 The operation being parallelized
heavy_month_summary <- function(dt) {
  dt <- copy(dt)
  dt[, tip_bracket := vapply(tip_pct, function(x) {
    if (is.na(x)) return(NA_character_)
    if (x == 0) "none" else if (x < 10) "low" else if (x < 20) "medium" else "high"
  }, character(1))]
  dt[, .(trips = .N, total_revenue = sum(total_amount), avg_fare = mean(fare_amount)),
     by = .(PUZone, DOZone, tip_bracket)]
}

# 4.3 Sequential
t_seq_start <- Sys.time()
seq_results <- lapply(month_splits, heavy_month_summary)
t_seq <- as.numeric(Sys.time() - t_seq_start, units = "secs")
cat(sprintf("Sequential total time: %.3f sec\n", t_seq))

# 4.4 Parallel (foreach + doParallel)
n_cores   <- max(1, parallel::detectCores() - 1)
n_workers <- min(n_cores, length(month_splits))
cat(sprintf("Detected cores: %d | workers used: %d\n", parallel::detectCores(), n_workers))

cl <- makeCluster(n_workers)
registerDoParallel(cl)

t_par_start <- Sys.time()
par_results <- foreach(m = month_splits, .packages = "data.table") %dopar% {
  heavy_month_summary(m)
}
t_par <- as.numeric(Sys.time() - t_par_start, units = "secs")
stopCluster(cl)
cat(sprintf("Parallel total time (%d workers): %.3f sec\n", n_workers, t_par))

# 4.5 Speedup
speedup <- t_seq / t_par
cat(sprintf("Sequential: %.3f sec | Parallel: %.3f sec | Speedup: %.2fx (on %d workers)\n",
            t_seq, t_par, speedup, n_workers))

identical_check <- all(mapply(function(a, b) isTRUE(all.equal(sum(a$total_revenue), sum(b$total_r
                              seq_results, par_results))
```

```
cat("Sequential and parallel results match:", identical_check, "\n")

# task 5

bench_results <- bench::mark(
  `Base R (aggregate)`      = aggregate(fare_amount ~ pickup_hour, data = trips, FUN = mean),
  `data.table (by=)`        = trips[, .(mean(fare_amount)), by = pickup_hour],
  `Functional (lapply)`     = avg_fare_lapply(hours_vec, fare_vec, hour_levels),
  `Functional (purrr::map)` = avg_fare_purrr(hours_vec, fare_vec, hour_levels),
  `Vectorized (tapply)`     = avg_fare_vectorized(hours_vec, fare_vec),
  iterations = 5, check = FALSE, filter_gc = FALSE
)
print(bench_results[, c("expression", "min", "median", "mem_alloc")])

seqpar_table <- data.frame(
  Approach = c("Sequential (lapply over months)",
               sprintf("Parallel (foreach/doParallel, %d workers)", n_workers)),
  Time_s   = round(c(t_seq, t_par), 3),
  Speedup  = c(1, round(speedup, 2))
)
print(seqpar_table)

ggplot(bench_results, aes(x = reorder(as.character(expression), as.numeric(median)),
                          y = as.numeric(median) * 1000)) +
  geom_col(fill = "#2C7FB8") +
  coord_flip() +
  labs(title = "Median execution time by R programming approach",
       subtitle = "Operation: average fare amount grouped by hour of day",
       x = NULL, y = "Median time (milliseconds)") +
  theme_minimal(base_size = 12)
ggsave(file.path(output_dir, "benchmark_comparison.png"), width = 8, height = 4, dpi = 120)

# task 6

plot_sample <- trips[sample(.N, min(.N, 50000))]

p1 <- ggplot(by_hour, aes(x = pickup_hour, y = trips)) +
  geom_col(fill = "#2C7FB8") + scale_x_continuous(breaks = 0:23) +
  labs(title = "Taxi Demand by Hour of Day", x = "Hour of day (0-23)", y = "Number of trips") +
  theme_minimal()

p2 <- ggplot(by_dow, aes(x = pickup_dow, y = trips, fill = pickup_dow)) +
  geom_col(show.legend = FALSE) +
  labs(title = "Taxi Demand by Day of Week", x = "Day of week", y = "Number of trips") +
  theme_minimal()

p3 <- ggplot(by_month, aes(x = pickup_month, y = trips, group = 1)) +
  geom_line(color = "#2C7FB8", linewidth = 1) + geom_point(size = 2, color = "#2C7FB8") +
  labs(title = "Monthly Taxi Demand", x = "Month", y = "Number of trips") +
  theme_minimal()

p4 <- ggplot(fare_by_hour, aes(x = pickup_hour, y = avg_fare)) +
  geom_line(color = "#D95F0E", linewidth = 1) + geom_point(color = "#D95F0E") +
  scale_x_continuous(breaks = 0:23) +
  labs(title = "Average Fare by Hour of Day", x = "Hour of day", y = "Average fare (USD)") +
  theme_minimal()

p5 <- ggplot(fare_by_hour, aes(x = pickup_hour, y = total_revenue / 1000)) +
  geom_area(fill = "#41AB5D", alpha = 0.6) + scale_x_continuous(breaks = 0:23) +
  labs(title = "Total Revenue by Hour of Day", x = "Hour of day", y = "Total revenue (thousand US
  theme_minimal()

top_pu <- trips[, .(trips = .N), by = PUZone][order(-trips)][1:10]
p6 <- ggplot(top_pu, aes(x = reorder(PUZone, trips), y = trips)) +
  geom_col(fill = "#2C7FB8") + coord_flip() +
  labs(title = "Top 10 Pickup Zones", x = NULL, y = "Number of trips") + theme_minimal()
```

```
top_do <- trips[, .(trips = .N), by = DOZone][order(-trips)][1:10]
p7 <- ggplot(top_do, aes(x = reorder(DOZone, trips), y = trips)) +
  geom_col(fill = "#D95F0E") + coord_flip() +
  labs(title = "Top 10 Drop-off Zones", x = NULL, y = "Number of trips") + theme_minimal()

top_routes[, route := paste(PUZone, "→", DOZone)]
p8 <- ggplot(top_routes, aes(x = reorder(route, trips), y = trips)) +
  geom_col(fill = "#756BB1") + coord_flip() +
  labs(title = "Top 10 Most Frequently Travelled Routes", x = NULL, y = "Number of trips") + then

top_rev_routes[, route := paste(PUZone, "→", DOZone)]
p9 <- ggplot(top_rev_routes, aes(x = reorder(route, total_revenue), y = total_revenue)) +
  geom_col(fill = "#E7298A") + coord_flip() +
  labs(title = "Top 10 Highest-Revenue Routes", x = NULL, y = "Total revenue (USD)") + theme_mini

p10 <- ggplot(payment_borough, aes(x = PUBorough, y = trips, fill = payment_type)) +
  geom_col(position = "fill") + scale_y_continuous(labels = scales::percent) +
  labs(title = "Payment Type Distribution by Borough", x = "Pickup borough",
       y = "Share of trips", fill = "Payment type") +
  theme_minimal() + theme(axis.text.x = element_text(angle = 30, hjust = 1))

p11 <- ggplot(plot_sample, aes(x = trip_distance, y = fare_amount)) +
  geom_hex(bins = 40) + scale_fill_viridis_c(name = "Trip count") +
  coord_cartesian(xlim = c(0, quantile(trips$trip_distance, 0.99)),
                  ylim = c(0, quantile(trips$fare_amount, 0.99))) +
  labs(title = "Trip Distance vs. Fare Amount", x = "Trip distance (miles)", y = "Fare amount (US
  theme_minimal()

plots <- list(p1=p1,p2=p2,p3=p3,p4=p4,p5=p5,p6=p6,p7=p7,p8=p8,p9=p9,p10=p10,p11=p11)
for (nm in names(plots)) {
  ggsave(file.path(output_dir, paste0(nm, ".png")), plots[[nm]], width = 8, height = 5, dpi = 12
}

# task 7

dt_time     <- as.numeric(bench_results$median[bench_results$expression == "data.table (by=)"])
agg_time    <- as.numeric(bench_results$median[bench_results$expression == "Base R (aggregate)"])
lapply_time <- as.numeric(bench_results$median[bench_results$expression == "Functional (lapply)"]
vec_time    <- as.numeric(bench_results$median[bench_results$expression == "Vectorized (tapply)"]

cat(sprintf("
1. Fastest approach: data.table (by=) — %.1fx faster than base aggregate().
2. data.table avoids copy-on-modify, uses C-level radix/hash grouping and
   reference-based joins — hence the memory and speed edge at scale.
3. Functional programming (lapply/purrr::map) wins on readability/composability
   for non-vectorizable, per-row logic (e.g. the tip_bracket classification
   in Task 4) — not on raw speed.
4. Vectorized tapply beat the lapply/split approach by %.1fx here, since it
   skips the memory-duplicating split() step for a single grouping variable.
5. Parallel speedup achieved: %.2fx on %d worker(s) (%d cores detected).
6. Parallel computing did NOT always help here — cluster start-up and
   serialization overhead can offset gains when partitions are few/small
   relative to core count (Amdahl's Law).
7. Trade-offs: data.table = fastest/leanest but less readable syntax;
   vectorized = fast + readable but limited to simple grouping; functional =
   most flexible/readable for irregular logic but slowest/most memory-hungry;
   parallel = real scalability only once per-task work clears fixed overhead.
8. Recommendation: data.table as the default engine for cleaning/joining/
   aggregation at this scale; purrr/lapply only for non-vectorizable logic;
   foreach/doParallel layered on top only once benchmarking shows the
   workload justifies the overhead.
", agg_time/dt_time, lapply_time/vec_time, speedup, n_workers, parallel::detectCores()))



length(par_results)
```

```
sapply(par_results, nrow)

# Compare against the sequential run from before (if seq_results is still in memory)
identical_check <- all(mapply(function(a, b) isTRUE(all.equal(sum(a$total_revenue), sum(b$total_r
                              seq_results, par_results))
cat("Sequential and parallel results match:", identical_check, "\n")

# Timing — re-run both with proper timers now that the cluster works
t_seq_start <- Sys.time()
seq_results <- lapply(month_splits, heavy_month_summary)
t_seq <- as.numeric(Sys.time() - t_seq_start, units = "secs")

cl <- makeCluster(n_workers, outfile = "")
registerDoParallel(cl)
clusterEvalQ(cl, library(data.table))
t_par_start <- Sys.time()
par_results <- foreach(m = month_splits, .packages = "data.table") %dopar% {
  heavy_month_summary(m)
}
t_par <- as.numeric(Sys.time() - t_par_start, units = "secs")
stopCluster(cl)

speedup <- t_seq / t_par
cat(sprintf("Sequential: %.3f sec | Parallel: %.3f sec | Speedup: %.2fx (on %d workers)\n",
            t_seq, t_par, speedup, n_workers))
```